

Mục lục

1 Bài 1: Đồng hồ	2
1.1 Đề bài	2
1.2 Lời giải	3
1.2.1 Nhận xét quan trọng	3
1.2.2 Subtask 1 (30% số điểm, $N, M, Q, a_i \leq 1000$)	3
1.2.3 Subtask 2 (30% số điểm, $N, M, Q \leq 1000$)	3
1.2.4 Subtask 3 (40% số điểm, Không có ràng buộc thêm) - Lời giải đầy đủ	4
2 Bài 2: Áo Ấm	8
2.1 Đề bài	8
2.2 Lời giải	8
2.2.1 Nhận xét quan trọng	8
2.2.2 Subtask 1 (25% số điểm, $N \leq 10$)	8
2.2.3 Subtask 2 (25% số điểm, $N \leq 300$)	9
2.2.4 Subtask 3 (25% số điểm, $N \leq 1500$)	9
2.2.5 Subtask 4 (25% số điểm, Không có ràng buộc thêm) - Lời giải đầy đủ	9
2.2.6 Phụ lục: Chứng minh độ phức tạp $O(N\sqrt{N})$ bằng BDT AM-GM	12
3 Bài 3: Tiệc trên núi	14
3.1 Đề bài	14
3.2 Lời giải	14
3.2.1 Nhận xét quan trọng	14
3.2.2 Subtask 1 (20% số điểm, $N, M, Q \leq 100$)	14
3.2.3 Subtask 2 (20% số điểm, $N, M, Q \leq 2000$)	15
3.2.4 Subtask 3 (20% số điểm, $r_1 = 1$ và $r_i = i - 1$ với $i \geq 2$)	15
3.2.5 Subtask 4 (20% số điểm, $M \leq 100$)	16
3.2.6 Subtask 5 (20% số điểm, không ràng buộc thêm) - Lời giải đầy đủ	16

Thông tin

Tác giả

- **Hoàng Hồ Minh Quang** - THPT Chuyên Bình Long, ngành Khoa Học Máy Tính trường Đại học Khoa Học Tự Nhiên Đại Học Quốc Gia Hồ Chí Minh

Đơn vị tài trợ

Trung tâm tin học. **Code Prep Pros** - là trung tâm luyện thi lập trình dành cho học sinh từ cấp tỉnh đến quốc gia, được thiết kế bởi các VOIer giàu kinh nghiệm. Chương trình cung cấp kiến thức vững chắc, kỹ năng giải quyết vấn đề và chiến lược làm bài tối ưu, giúp học sinh tự tin chinh phục các kỳ thi lập trình.

Group nộp bài, trao đổi và sửa đề thi Tin Học Trẻ, HSG Tỉnh, Chọn DTQG, vào chuyên tin 10

Tham gia ngay cộng đồng chia sẻ tài liệu, đề thi và lời giải cho kỳ thi vào lớp 10 chuyên Tin, Tin Học Trẻ bảng B - C, học sinh giỏi Tỉnh và học sinh giỏi Quốc Gia. Cơ hội rèn luyện, học hỏi từ tất cả các bạn trên Cả nước. **Bấm vào đây!**

§1 Bài 1: Đồng hồ

§1.1 Đề bài

Thầy Nam mua N chiếc đồng hồ ở hành tinh An Bình, nơi một ngày có đúng M giờ (từ 0 đến $M - 1$). Mỗi chiếc đồng hồ thứ i có đặc điểm sau:

- Đang hiển thị giờ b_i tại thời điểm mua.
- Sẽ hoạt động thêm đúng a_i giờ nữa rồi dừng hẳn. (Mỗi giờ trôi qua, số hiển thị tăng thêm 1, sau $M - 1$ sẽ quay về 0).

Cho Q truy vấn, mỗi truy vấn là một số t_i . Yêu cầu tính tổng số lần giờ t_i xuất hiện trên màn hình của tất cả N chiếc đồng hồ trước khi chúng dừng hoạt động.

Giới hạn dữ liệu

- $1 \leq N, Q \leq 2 \cdot 10^5$
- $1 \leq M \leq 10^9$
- $1 \leq a_i \leq 10^{12}$
- $0 \leq b_i < M$ và $0 \leq t_i < M$

Các Subtask

- **Subtask 1** (30% số điểm): $N, M, Q, a_i \leq 1000$
- **Subtask 2** (30% số điểm): $N, M, Q \leq 1000$
- **Subtask 3** (40% số điểm): Không có ràng buộc thêm

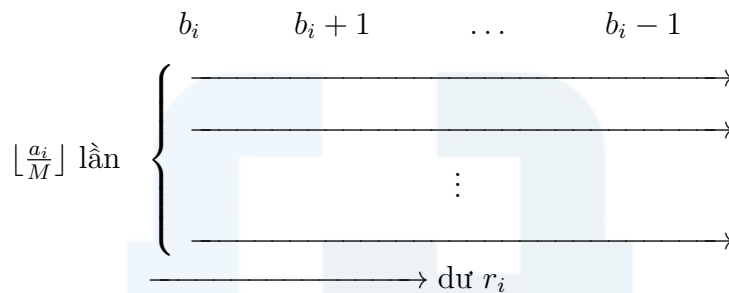
CODE PREP PROS
EXPERIENCE-EFFORT-ACHIEVEMENT

§1.2 Lời giải

§1.2.1 Nhận xét quan trọng

Ta quan sát quá trình chạy của một chiếc đồng hồ i :

- Bắt đầu từ b_i , sau k giờ (với $0 \leq k < a_i$), giờ hiển thị trên đồng hồ sẽ là $(b_i + k) \pmod{M}$.
- Vì một chu kỳ đồng hồ có độ dài đúng M , nên nếu $a_i \geq M$, chiếc đồng hồ này sẽ chạy qua toàn bộ các giờ từ 0 đến $M - 1$ đúng $\lfloor \frac{a_i}{M} \rfloor$ lần trong các chu kỳ đầy đủ.
- Sau khi bỏ đi các chu kỳ đầy đủ, đồng hồ sẽ chạy thêm một số bước dư là $r_i = a_i \pmod{M}$. Các bước dư này sẽ tạo thành một đoạn liên tiếp trên một mảng vòng tròn (circular array) bắt đầu từ b_i .



Điều này cho ta định hướng: Tách riêng phần "chu kỳ đầy đủ" (ai cũng được cộng số lần như nhau) và phần "dư" (chỉ một số giờ cụ thể được cộng thêm).

§1.2.2 Subtask 1 (30% số điểm, $N, M, Q, a_i \leq 1000$)

¶ **Tiếp cận.** Do các giới hạn đều rất nhỏ, ta có thể mô phỏng trực tiếp quá trình chạy của từng chiếc đồng hồ. Ta dùng một mảng đếm `cnt` kích thước M , với `cnt[t]` lưu số lần giờ t xuất hiện. Với mỗi đồng hồ i , ta cho một biến `cur` chạy a_i lần bắt đầu từ b_i , mỗi lần tăng `cnt[cur]` lên 1 và cập nhật `cur = (cur + 1) % M`.

¶ **Chứng minh tính đúng đắn.** Cách làm này bám sát hoàn toàn định nghĩa của đề bài, mô phỏng đúng từng giờ trôi qua của từng đồng hồ nên kết quả luôn chính xác.

¶ Độ phức tạp.

- Thời gian: $O(\sum a_i + Q)$, vì ta chạy vòng lặp mô phỏng cho từng đồng hồ.
- Bộ nhớ: $O(M)$ để lưu mảng `cnt`.

§1.2.3 Subtask 2 (30% số điểm, $N, M, Q \leq 1000$)

¶ **Ý tưởng / Tiếp cận.** Ở subtask này a_i có thể lên tới 10^{12} , nên việc lặp a_i lần như Subtask 1 sẽ gây quá thời gian (TLE). Nhắc lại nhận xét quan trọng, ta có thể phân tích:

$$a_i = q_i \cdot M + r_i$$

Trong đó $q_i = \lfloor \frac{a_i}{M} \rfloor$ và $r_i = a_i \pmod{M}$. Mỗi giờ từ 0 đến $M - 1$ đều sẽ chắc chắn được xuất hiện q_i lần. Do đó, ta tính tổng tất cả các q_i của mọi đồng hồ vào một biến `base`.

Với phần dư r_i , vì $0 \leq r_i < M \leq 1000$, ta hoàn toàn có thể chạy vòng lặp mô phỏng r_i bước giống như Subtask 1 để cộng vào một mảng `extra`. Kết quả cho truy vấn t sẽ là `base + extra[t]`.

¶ **Chứng minh tính đúng đắn.** Việc tách chu kỳ dựa trên tính chất tuần hoàn của phép chia lấy dư. Tổng số lần xuất hiện của một số là tổng **số lần ở các chu kỳ lặp lại** cộng với **phần dư chưa hết một vòng**, do đó ta đảm bảo không bỏ sót bất kỳ trạng thái nào.

¶ **Độ phức tạp.**

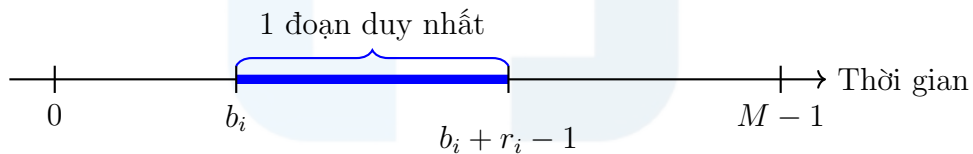
- Thời gian: $O(N \cdot M + Q)$, vì phần dư $r_i < M$.
- Bộ nhớ: $O(M)$ cho mảng tần số.

§1.2.4 Subtask 3 (40% số điểm, Không có ràng buộc thêm) - Lời giải đầy đủ

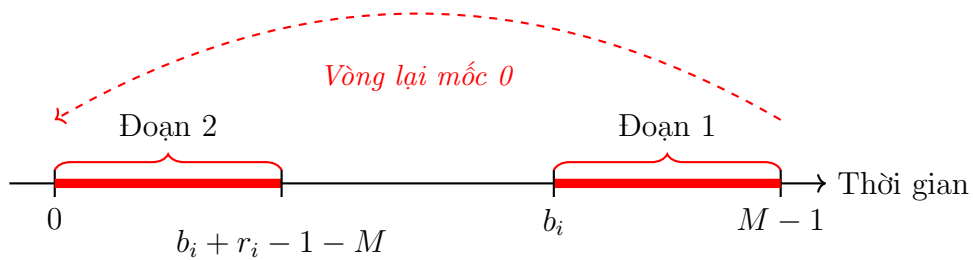
¶ **Ý tưởng / Tiếp cận.** Với $M \leq 10^9$, ta không thể tạo mảng `extra` kích thước M , và thời gian $O(N \cdot M)$ cũng sẽ gây TLE. Làm sao để xử lý phần dư r_i một cách tối ưu?

¶ **Nhận xét:** Ta thấy r_i bước dư chỉ tạo thành một đoạn trên vòng tròn giờ. Cụ thể, các giờ được cộng thêm 1 lần nằm trong đoạn $[b_i, (b_i + r_i - 1) \pmod{M}]$.

- **Không vòng qua mốc 0:** Tạo thành 1 đoạn $[b_i, b_i + r_i - 1]$.



- **Vòng qua mốc 0:** Bị cắt thành 2 đoạn là $[b_i, M - 1]$ và $[0, b_i + r_i - 1 - M]$.



¶ **Bài toán quy về:** Cho một tập hợp các đoạn $[L, R]$, đếm xem một điểm t bị bao phủ bởi bao nhiêu đoạn. Cấu trúc dữ liệu và thuật toán phù hợp nhất ở đây chính là Đường quét (Sweep Line).

¶ Các bước của thuật toán.

1. Tính **base** là tổng các $\lfloor \frac{a_i}{M} \rfloor$ của mọi đồng hồ.
2. Với mỗi đồng hồ, xét $r_i = a_i \pmod{M}$. Nếu $r_i > 0$, ta sinh ra các đoạn bị bao phủ như đã phân tích ở trên.
3. Áp dụng kỹ thuật Sweep Line. Mỗi đoạn $[l, r]$ sẽ sinh ra 2 sự kiện (event):
 - Bắt đầu đoạn: Tọa độ l , loại sự kiện là $+1$.
 - Kết thúc đoạn: Tọa độ r , loại sự kiện là -1 .

Mỗi truy vấn t mang chỉ số id được xem như một sự kiện hỏi: Tọa độ t , loại sự kiện là 0.

4. Sắp xếp các sự kiện: Sắp xếp tăng dần theo tọa độ. Nếu trùng tọa độ, ta phải ưu tiên như sau: xử lý sự kiện mở đoạn $(+1) \rightarrow$ sau đó mới đến trả lời sự kiện truy vấn $(0) \rightarrow$ và cuối cùng là sự kiện đóng đoạn (-1) . Việc này đảm bảo cả hai mút l và r của đoạn đều được tính vào truy vấn tại đúng vị trí đó.
5. Duyệt qua mảng sự kiện đã sắp xếp, duy trì biến **active** là số lượng đoạn đang bao phủ. Khi gặp sự kiện hỏi, kết quả cho truy vấn đó là **base + active**.

¶ **Chứng minh tính đúng đắn.** Thuật toán Đường quét tính toán trực tiếp mảng cộng dồn (prefix sum) của mảng hiệu (difference array) nhưng chỉ trên những tọa độ có sự thay đổi. Trật tự sắp xếp sự kiện $+1 \rightarrow 0 \rightarrow -1$ tại cùng một tọa độ xử lý hoàn hảo bài toán bao gồm cả hai mút của các đoạn đóng $[l, r]$.

¶ **Ví dụ minh họa.** Giả sử có đoạn bao phủ là $[3, 5]$, và ta cần truy vấn tại các điểm 3, 5, 6.

- Sự kiện đoạn: $(3, +1)$ và $(5, -1)$.
- Sự kiện truy vấn: $(3, 0), (5, 0), (6, 0)$.

Dãy sự kiện sau khi sắp xếp:

- Tại tọa độ 3: $(3, +1) \rightarrow (3, 0)$. Biến **active** tăng thành 1, truy vấn lấy kết quả 1.
- Tại tọa độ 5: $(5, 0) \rightarrow (5, -1)$. Biến **active** vẫn là 1 nên truy vấn lấy kết quả 1, sau đó **active** giảm về 0.
- Tại tọa độ 6: $(6, 0)$. Biến **active** đang là 0, truy vấn lấy kết quả 0.

Kịch bản chạy ra kết quả hoàn toàn chính xác!

¶ **Lưu ý về kiểu dữ liệu.** Biến **base** là tổng của $\frac{10^{12}}{10^9} \cdot 2 \cdot 10^5$, có thể đạt giá trị $2 \cdot 10^{17}$, do vậy bắt buộc phải dùng kiểu số nguyên 64-bit (`long long` trong C++). Mảng lưu kết quả **ans** cũng phải dùng `long long`.

¶ Độ phức tạp.

- Mỗi đồng hồ tạo tối đa 4 sự kiện, mỗi truy vấn tạo 1 sự kiện. Tổng số sự kiện là $O(N + Q)$.
- Do ta cần sort tất cả các sự kiện, nên thời gian chạy thuật toán là $O((N + Q) \log(N + Q))$.
- Không gian bộ nhớ tiêu tốn để chứa mảng sự kiện và lưu kết quả là $O(N + Q)$.

¶ Mã nguồn minh họa (C++).

```
1 #include <bits/stdc++.h> //NeOWami
2 using namespace std;
3
4 const int N = 2e5 + 5;
5 int n, m, q;
6 long long ans[N];
7
8 struct Event {
9     long long pos;
10    int type;
11    int id;
12
13    bool operator<(const Event& other) const {
14        if (pos != other.pos)
15            return pos < other.pos;
16        return type > other.type;
17    }
18 };
19
20 signed main() {
21     cin.tie(NULL)->sync_with_stdio(false);
22     if (ifstream("CLOCKS.inp")) {
23         freopen("CLOCKS.inp", "r", stdin);
24         freopen("CLOCKS.out", "w", stdout);
25     }
26     cin >> n >> m;
27
28     long long base_count = 0;
29     vector<Event> events;
30
31     for (int i = 0; i < n; i++) {
32         long long a, b;
33         cin >> a >> b;
34
35         base_count += (a / m);
36         long long r = a % m;
37
38         if (r > 0) {
39             long long left = b;
40             long long right = b + r - 1;
41
42             if (right < m) {
```

```
43         events.push_back({left, 1, -1});
44         events.push_back({right, -1, -1});
45     } else {
46         events.push_back({left, 1, -1});
47         events.push_back({m - 1, -1, -1});
48
49         events.push_back({0, 1, -1});
50         events.push_back({right - m, -1, -1});
51     }
52 }
53 }
54
55 cin >> q;
56 for (int i = 0; i < q; i++) {
57     long long t;
58     cin >> t;
59     events.push_back({t, 0, i});
60 }
61
62 sort(events.begin(), events.end());
63
64 long long active = 0;
65
66 for (const auto& ev : events) {
67     if (ev.type == 1) {
68         active++;
69     } else if (ev.type == -1) {
70         active--;
71     } else {
72         ans[ev.id] = base_count + active;
73     }
74 }
75
76 for (int i = 0; i < q; i++) {
77     cout << ans[i] << "\n";
78 }
79
80 return 0;
81 }
```

§2 Bài 2: Áo Ấm

§2.1 Đề bài

An cần di chuyển từ sân bay (vị trí 0) đến khách sạn (vị trí N). Quảng đường dài N mét, tại mỗi vị trí i ($1 \leq i \leq N - 1$) có một cửa hàng quần áo. Khi đi ngang qua cửa hàng thứ i , An có thể:

- Tiếp tục đi (không tốn phí).
- Sưởi ấm (tốn 1 đồng, không mua áo).
- Sưởi ấm và mua thêm một lớp áo (tốn $1 + A_i$ đồng).

Ràng buộc di chuyển: An không được đi bộ liên tục vượt quá L mét mà không vào cửa hàng nào, với L là số lớp áo đang mặc trên người. Ban đầu ở vị trí 0, An có sẵn $L = 1$ lớp áo. Khách sạn ở vị trí N cũng phải tuân thủ điều kiện này (nghĩa là quãng đường từ cửa hàng cuối cùng ghé vào cho đến N không được vượt quá L).

Yêu cầu: Tìm số tiền ít nhất An cần trả để đến được khách sạn.

Giới hạn dữ liệu: $N \leq 10^5$, $A_i \leq 10^9$.

§2.2 Lời giải

§2.2.1 Nhận xét quan trọng

Bản chất của bài toán này là Quy hoạch động (Dynamic Programming) kết hợp với đường đi ngắn nhất. Trạng thái của An phụ thuộc vào 3 yếu tố: vị trí hiện tại i , số lớp áo đang mặc L , và số mét đã đi kể từ lần sưởi ấm gần nhất.

Ta thấy rằng mỗi lần quyết định mua thêm áo, An phải trả một mức phí tối thiểu là 2 đồng (1 đồng sưởi + ít nhất 1 đồng mua áo). Mặc dù mua áo giúp "mở rộng" giới hạn di chuyển L , nhưng nếu mua quá nhiều, chi phí sẽ rất cao. Việc cân bằng giữa số lần sưởi ấm định kỳ và số lớp áo mua thêm chính là mấu chốt để tối ưu hóa bài toán.

§2.2.2 Subtask 1 (25% số điểm, $N \leq 10$)

¶ **Tiếp cận.** Ta dùng Thuật toán Tìm kiếm theo chiều sâu (DFS) kết hợp Quay lui (Backtracking). Ta duy trì trạng thái (p, L, c) trong đó p là vị trí hiện tại, L là số lớp áo, và c là tổng chi phí đã trả. Từ vị trí p , cửa hàng tiếp theo An có thể ghé vào là mọi vị trí $i \in [p + 1, \min(N - 1, p + L)]$. Tại cửa hàng i , ta rẽ ra hai nhánh đệ quy: nhánh chỉ sưởi ấm chuyển trạng thái thành $(i, L, c + 1)$, và nhánh mua áo chuyển trạng thái thành $(i, L + 1, c + 1 + A_i)$. Nếu $N - p \leq L$, ta có thể đi thẳng tới khách sạn và cập nhật đáp án.

¶ **Chứng minh tính đúng đắn.** Việc rẽ nhánh vét cạn mọi kịch bản di chuyển hợp lệ theo đúng luật của bài toán, đảm bảo không bỏ sót bất kỳ tổ hợp hành động nào. Kết hợp với Kỹ thuật Nhánh cận (Branch and Bound): dừng đệ quy ngay khi chi phí $c \geq ans$ (ans là kỷ lục tốt nhất hiện tại), ta luôn lấy được đáp án đúng.

¶ **Độ phức tạp.** Thời gian là $O(C^N)$ (hàm mũ) do vét cạn mọi tập con các trạng thái. Không gian là $O(N)$ cho độ sâu của Stack đệ quy. Chạy hoàn toàn an toàn với $N \leq 10$.

§2.2.3 Subtask 2 (25% số điểm, $N \leq 300$)

¶ **Tiếp cận.** Để tránh duyệt lại, ta dùng Quy hoạch động theo từng mét. Gọi $dp[L][d]$ là chi phí nhỏ nhất khi An có L lớp áo và đã đi d mét kể từ lần vào cửa hàng gần nhất. Từ vị trí hiện tại có cửa hàng i , ta có các chuyển trạng thái:

- đi bộ tiếp: chuyển sang $(L, d + 1)$ với điều kiện $d + 1 \leq L$
- thao tác sưởi ấm: chuyển sang $(L, 1)$ với phí $+1$
- hoặc sưởi và mua áo: chuyển sang $(L + 1, 1)$ với phí $+1 + A_i$

¶ **Độ phức tạp.** Tại mỗi mét i , ta có $O(N^2)$ trạng thái (L, d) . Tổng thời gian là $O(N^3)$ và bộ nhớ là $O(N^2)$, phù hợp với $N \leq 300$.

§2.2.4 Subtask 3 (25% số điểm, $N \leq 1500$)

¶ **Tiếp cận.** Ta nhận ra thay vì đi từng mét, ta có thể "nhảy" giữa các trạm dừng. Đặt $f_L[i]$ là chi phí nhỏ nhất khi ở vị trí i ngay sau khi mua lớp áo thứ L . Đặt $g_L[i]$ là chi phí nhỏ nhất khi An vừa sưởi ấm tại cửa hàng i nhưng chưa mua áo. Để dễ xử lý điểm cuối, ta coi khách sạn N là cửa hàng ảo có phí sưởi 1 (sẽ trừ đi 1 ở đáp án cuối cùng). Vị trí dừng gần nhất j trước khi tới i phải thỏa mãn $\max(0, i - L) \leq j < i$. Ta có công thức chuyển:

$$g_L[i] = 1 + \min_{\max(0, i-L) \leq j < i} \min(f_L[j], g_L[j])$$

$$f_{L+1}[i] = g_L[i] + A_i \quad (\text{với } i < N)$$

¶ **Chứng minh tính đúng đắn.** Việc gom trạng thái mua áo (f_L) và sưởi (g_L) đại diện đầy đủ cho quyết định tại một trạm. Sử dụng hai Cây phân đoạn (Segment Tree) cho yêu cầu Range Minimum Query để lấy min trên đoạn độ dài L tốn $O(\log N)$ mỗi bước.

¶ **Độ phức tạp.** Có tối đa N giá trị lớp L , mỗi lớp duyệt N phần tử, mỗi bước truy vấn mất $O(\log N)$. Thời gian là $O(N^2 \log N)$, không gian $O(N)$ do chỉ cần lưu mảng 2 lớp áo liên kề.

§2.2.5 Subtask 4 (25% số điểm, Không có ràng buộc thêm) - Lời giải đầy đủ

¶ **Ý tưởng / Tiếp cận.** Tại sao $O(N^2 \log N)$ vẫn chưa đủ tốt với $N \leq 10^5$? Có hai điểm ta có thể tối ưu vượt bậc:

- Làm sao tính đoạn min nhanh hơn? Ta có thể giảm từ $O(\log N)$ xuống $O(1)$ bằng Hàng đợi đơn điệu (Monotonic Deque). Hơn nữa, thay vì quét j cho biến g_L , nếu $i - p > L$ (với p là trạm mua áo gần nhất), số lần sưởi là $\lceil (i - p) / L \rceil = 1 + \lceil (i - L - p) / L \rceil$. Phép toán này cho phép ta tái sử dụng trực tiếp trạng thái cũ: $g_L[i] = 1 + \min(\min f_L[j], g_L[i - L])$.
- Tại sao phải tính tới lớp áo thứ N ? Với mỗi lớp áo, chi phí tăng tối thiểu 2. Nếu L rất lớn, việc sưởi định kỳ với chi phí thấp còn rẻ hơn mua áo. Thực tế, sau khoảng $\approx \sqrt{N}$ lớp áo đầu tiên, số lần sưởi giảm đi cực ít. Bằng cách dùng kỹ thuật Nhánh cận (Branch and Bound), khi chi phí để đạt được lớp L vượt qua kỷ lục ans đang có, ta sẽ loại bỏ nhánh đó. Điều này giới hạn số lớp áo có ý nghĩa ở mức cực kỳ thấp. (Chứng minh ở Phụ lục bên dưới)

¶ Các bước của thuật toán.

1. Khởi tạo mảng $f[0] = 0$ tương ứng với lớp áo $L = 1$ ở sân bay. Đặt $ans = \infty$.
2. Duyệt từng lớp áo L từ $1 \dots N$. Tạo mảng g (lưu chi phí sưởi tại lớp L) và $next_f$ (lưu chi phí nhảy sang $L + 1$).
3. Tại mỗi i từ $0 \dots N$, dùng một deque để duy trì phần tử có $f[j]$ nhỏ nhất trong "cửa sổ trượt" kích thước L là đoạn $[i - L, i - 1]$.
4. Tính $g[i]$ bằng cách lấy $1 + \min(\text{giá trị min trong cửa sổ}, g[i - L])$.
5. Nếu $i = N$, cập nhật $ans = \min(ans, g[N] - 1)$. Nếu $i < N$ và $g[i] + A_i < ans$, cập nhật $next_f[i] = g[i] + A_i$.
6. Nếu giá trị tối thiểu của lớp L hiện tại lớn hơn hoặc bằng ans , dừng vòng lặp lớn (Break). Sau đó gán $f = next_f$ để tiếp tục lớp $L + 1$.

¶ **Chứng minh tính đúng đắn.** Việc sử dụng $g[i - L]$ bao hàm hệ quả “nếu đi nhiều hơn một đoạn L mét thì ta tốn thêm đúng 1 đồng sưởi tại $i - L$ trước khi sang trạm i ”. Phương pháp cắt tỉa (pruning) đảm bảo thuật toán không bỏ sót hành trình tối ưu nào, vì mọi trạng thái bị bỏ qua đều được chứng minh bằng mặt toán học ở “Phụ lục” là đắt đỏ hơn giá trị kỷ lục ta đã tìm thấy.

¶ **Ví dụ minh họa.** Xét test mẫu thứ nhất với $N = 8, A = [5, 1, 9, 1, 9, 9, 9]$. Tại $L = 1$:

- Ban đầu ở vị trí 0, $f[0] = 0$.
- Khi tới cửa hàng $i = 1$: cửa sổ $[0, 0]$ duy nhất giá trị $f[0] = 0$. Vậy $g[1] = 1 + 0 = 1$. Chi phí để mua áo thứ hai tại đây là $next_f[1] = g[1] + A_1 = 1 + 5 = 6$.
- Khi tới $i = 2$: $g[2] = 1 + \min(f[1], g[1]) = 1 + \min(\infty, 1) = 2$. Chi phí mua áo $next_f[2] = g[2] + A_2 = 2 + 1 = 3$.
- Quá trình tính tiến mảng này diễn ra như một nhịp sóng lan dần, và thuật toán nhanh chóng tìm được $next_f[4] = 5$, và đi thẳng đến $N = 8$ với đáp án 5.

¶ **Lưu ý về kiểu dữ liệu.** Giới hạn giá áo A_i lên tới 10^9 . Nếu cộng dồn liên tiếp, tổng chi phí có thể vượt quá giới hạn 2×10^9 của kiểu số nguyên thông thường (Overflow 32-bit). Bắt buộc phải khởi tạo giá trị vô cùng (infinity) $\geq 10^{18}$ và dùng kiểu long long để lưu chi phí.

¶ Mã nguồn minh họa (C++).

```
1 #include <bits/stdc++.h> //NeOWami
2 using namespace std;
3
4 const int N = 1e5 + 5;
5 const long long INF = 1e18;
6 int n;
7 long long a[N], f[N], g[N], next_f[N];
8
9 signed main() {
10     cin.tie(NULL)->sync_with_stdio(false);
```

```
11     if (ifstream("COATS.INP")) {
12         freopen("COATS.INP", "r", stdin);
13         freopen("COATS.OUT", "w", stdout);
14     }
15     cin >> n;
16
17     for (int i = 1; i < n; ++i) cin >> a[i];
18
19     fill(f, f + n + 1, INF);
20     f[0] = 0;
21     long long ans = INF;
22
23     for (int L = 1; L <= n; ++L) {
24         fill(g, g + n + 1, INF);
25         fill(next_f, next_f + n + 1, INF);
26         deque<int> dq;
27         long long min_f_layer = INF;
28
29         for (int i = 0; i <= n; ++i) {
30             if (f[i] < INF) min_f_layer = min(min_f_layer, f[i]);
31
32             while (!dq.empty() && dq.front() < i - L) {
33                 dq.pop_front();
34             }
35
36             if (i > 0) {
37                 int j = i - 1;
38                 while (!dq.empty() && f[dq.back()] >= f[j]) {
39                     dq.pop_back();
40                 }
41                 dq.push_back(j);
42             }
43
44             long long min_f_window = INF;
45             if (!dq.empty()) {
46                 min_f_window = f[dq.front()];
47             }
48
49             if (i > 0) {
50                 if (min_f_window < INF) {
51                     g[i] = min(g[i], min_f_window + 1);
52                 }
53                 if (i - L >= 0 && g[i - L] < INF) {
54                     g[i] = min(g[i], g[i - L] + 1);
55                 }
56             }
57
58             if (i == n) {
59                 if (g[n] < INF) ans = min(ans, g[n] - 1);
60             } else if (i > 0) {
61
62                 if (g[i] < INF && g[i] + a[i] < ans) {
63                     next_f[i] = g[i] + a[i];
```

```

64         }
65     }
66 }
67
68
69     if (min_f_layer >= ans) break;
70
71     swap(f, next_f);
72
73     bool alive = false;
74     for (int i = 0; i <= n; ++i) {
75         if (f[i] < INF) { alive = true; break; }
76     }
77     if (!alive && g[n] == INF) break;
78 }
79
80 cout << ans << "\n";
81 return 0;
82 }

```

¶ Độ phức tạp.

- Bằng cách dùng hàng đợi đơn điệu, mỗi phần tử được push và pop nhiều nhất một lần, nên vòng lặp bên trong xử lý mất $O(N)$. Cộng với dùng Nhánh cận (Branch and Bound), sau mốc $L \approx \sqrt{N}$, số lớp áo tồn tại để tạo ra trạng thái tốt hơn ans giảm mạnh theo hàm $O(N/L)$. Do đó tổng số vòng lặp bên ngoài bị giới hạn. Độ phức tạp thời gian tổng quát là $O(N\sqrt{N})$.
- Bộ nhớ giảm xuống $O(N)$ nhờ chỉ lưu giữ trạng thái của duy nhất hai lớp áo kề nhau trong quá trình QHĐ.

§2.2.6 Phụ lục: Chứng minh độ phức tạp $O(N\sqrt{N})$ bằng BĐT AM-GM

Mệnh đề: Với kỹ thuật Nhánh cận (Branch and Bound), số lượng lớp áo (trạng thái L) mà thuật toán thực sự cần xử lý không bao giờ vượt quá $O(\sqrt{N})$.

Thay vì đếm chi tiết từng bước chuyển trạng thái, ta có thể chứng minh nhanh giới hạn này thông qua điểm cực tiểu của hàm chi phí tổng quát. Đánh giá tổng chi phí để An đi hết N mét với L lớp áo, ta thấy chi phí này luôn được cấu thành từ hai phần chính:

- **Chi phí mua áo (tăng bậc nhất theo L):** Để sở hữu L lớp áo, An phải mua thêm áo. Mỗi lần mua tốn tối thiểu 2 đồng (1 đồng sườn + ít nhất 1 đồng giá áo). Tổng chi phí mua áo tối thiểu là $2L$.
- **Chi phí sườn định kỳ (tỉ lệ nghịch với L):** Với L lớp áo, khoảng cách đi tối đa giữa hai lần sườn là L . Để đi hết quãng đường N , số lần dừng sườn định kỳ (nếu không mua thêm áo) xấp xỉ $\frac{N}{L}$. Mỗi lần sườn tốn 1 đồng, vậy chi phí sườn là $\frac{N}{L}$.

Gọi $f(L)$ là ước lượng chi phí tối thiểu, ta có hàm mục tiêu:

$$f(L) \approx 2L + \frac{N}{L}$$

Áp dụng Bất đẳng thức AM-GM (Cauchy) cho hai đại lượng dương, ta có:

$$f(L) = 2L + \frac{N}{L} \geq 2\sqrt{2L \cdot \frac{N}{L}} = 2\sqrt{2N}$$

Nhận xét cốt lõi: Theo nguyên lý AM-GM, hàm chi phí này đạt giá trị cực tiểu (dấu "=" xảy ra) khi và chỉ khi hai đại lượng cân bằng nhau:

$$2L = \frac{N}{L} \implies L^2 = \frac{N}{2} \implies L = O(\sqrt{N})$$

Kết luận: Hàm chi phí $f(L)$ có đồ thị dạng hình chữ U, chạm đáy cực tiểu tại vùng $L \approx \sqrt{N}$. Khi vòng lặp tăng L vượt qua ngưỡng này, phần chi phí mua áo ($2L$) sẽ phình to và lấn át hoàn toàn lợi ích của việc giảm số lần sưởi ($\frac{N}{L}$). Lúc này, chi phí tối thiểu của nhánh chắc chắn lớn hơn kỷ lục *ans* đã được tìm thấy ở vùng đáy.

Nhờ điều kiện cắt tỉa Branch and Bound, thuật toán sẽ nhận diện sự "đắt đỏ" vô lý này và lập tức ngắt vòng lặp. Do đó, ta chỉ cần xử lý tối đa $O(\sqrt{N})$ lớp áo. Kết hợp với hàng đợi đơn điệu $O(N)$ tại mỗi lớp, tổng độ phức tạp thời gian được đảm bảo là $O(N\sqrt{N})$.



§3 Bài 3: Tiệc trên núi

§3.1 Đề bài

Ở thị trấn Lạc Sơn có N căn nhà (đánh số $1 \dots N$) và M người dân (đánh số $1 \dots M$). Mỗi người sống trong đúng một nhà, mỗi nhà có tối đa một người. Các căn nhà nằm trên sườn núi, với mỗi nhà i có một đường trượt một chiều tới nhà r_i . Có đúng một nhà s mà $r_s = s$ (chân núi); các đường trượt tạo thành một cây gốc tại s . Người dân quen nhau theo vòng tròn: người i quen $i - 1$ và $i + 1$ (vòng quanh 1 và M). Có Q sự kiện:

- Loại 1 $a \ b$: đổi chỗ cư dân của hai nhà a và b (nếu một nhà trống thì chuyển người sang nhà trống).
- Loại 2 $p \ R$: người p muốn tổ chức tiệc, chọn một hướng trên vòng tròn quen biết và mời một đoạn liên tiếp bắt đầu từ p (tức p là một đầu của đoạn khách mời). Nhóm khách hợp lệ nếu tồn tại một nhà làm địa điểm sao cho mọi người trong nhóm đi từ nhà hiện tại tới đó không quá R đường trượt.

Yêu cầu: Với mỗi sự kiện loại 2, in ra số người lớn nhất có thể mời.

Giới hạn: $1 \leq M \leq N \leq 10^5$, $1 \leq Q \leq 10^5$, $1 \leq R < N$.

§3.2 Lời giải

§3.2.1 Nhận xét quan trọng

Xét một nhóm khách đang ở các nhà $X = \{x_1, x_2, \dots, x_k\}$. Muốn mọi người cùng đến được một địa điểm chung thì điểm đó phải là tổ tiên chung của tất cả các nhà trong X . Để khoảng cách lớn nhất từ các nhà tới điểm hẹn là nhỏ nhất, địa điểm tối ưu chính là **LCA** (tổ tiên chung thấp nhất) của tập X .

Khi đó, khoảng cách từ một nhà x_i đến $l = LCA(X)$ là $depth(x_i) - depth(l)$. Nhóm hợp lệ khi và chỉ khi:

$$\max_i depth(x_i) - depth(l) \leq R.$$

Khi ta thêm một người (thêm một nhà) vào nhóm:

- $\max depth$ không giảm (chỉ có thể tăng hoặc giữ nguyên).
- $depth(l)$ không tăng (LCA của tập lớn hơn là tổ tiên của LCA cũ, tức nằm gần gốc hơn hoặc bằng).

Vì vậy hiệu $\max depth - depth(l)$ là **không giảm** khi mở rộng nhóm. Nói cách khác, tính hợp lệ của đoạn khách mời có tính **đơn điệu**: nếu đoạn độ dài K (bắt đầu từ p) hợp lệ thì mọi đoạn ngắn hơn cũng hợp lệ; nếu đoạn độ dài K không hợp lệ thì mọi đoạn dài hơn cũng không hợp lệ.

Nhờ tính đơn điệu, thay vì thử tất cả độ dài, ta có thể tìm **biên xa nhất** hợp lệ của từng hướng, rồi lấy max số người.

§3.2.2 Subtask 1 (20% số điểm, $N, M, Q \leq 100$)

¶ **Tiếp cận.** Với mỗi truy vấn loại 2, ta thử lần lượt mở rộng đoạn theo hai hướng (cùng chiều kim đồng hồ và ngược lại). Với mỗi độ dài, duyệt lại toàn bộ các nhà trong đoạn đang xét để tính LCA, $\max depth$ và kiểm tra điều kiện.

- LCA của hai đỉnh được tìm bằng cách đưa đỉnh có độ sâu sâu hơn lên nút cha đến khi bằng độ sâu của đỉnh còn lại, sau đó đưa đồng thời cả hai lên nút cha đến khi gặp nhau.
- LCA của nhiều đỉnh được tính tuần tự: $l \leftarrow LCA(l, x_i)$.
- Trong lúc duyệt, nếu điều kiện bị vi phạm thì dừng hướng đó.
- Sự kiện đổi nhà chỉ cần cập nhật mảng ánh xạ người $\leftrightarrow$ nhà trong $O(1)$.

¶ Độ phức tạp.

- Mỗi truy vấn thử tối đa $O(M)$ độ dài, mỗi lần duyệt $O(M)$ nhà, mỗi LCA tốn $O(N)$. Tổng $O(Q \cdot M^2 \cdot N) \approx 10^2 \cdot 10^2 \cdot 10^2 = 10^6$ thao tác.

§3.2.3 Subtask 2 (20% số điểm, $N, M, Q \leq 2000$)

¶ **Tiếp cận.** Dùng **bitset** để tăng tốc tính LCA của nhiều đỉnh. Với mỗi đỉnh u , tạo một **bitset** $anc[u]$ độ dài N : bit thứ v bằng 1 nếu v là tổ tiên của u . Khi thêm nhà x_i vào đoạn, ta chỉ cần $common \leftarrow common \& anc[x_i]$; $mx \leftarrow \max(mx, depth(x_i))$. Các bit 1 trong $common$ là tập các tổ tiên chung, bit cao nhất (gần lá nhất) chính là LCA. Số bit 1 của $common$ trừ đi 1 cho ta $depth(lca)$.

Việc tiền xử lý $anc[u]$ có thể thực hiện bằng DFS: $anc[u] = anc[parent] \cup \{u\}$, biểu diễn bằng **bitset**.

¶ **Chứng minh tính đúng đắn.** Phép AND **bitset** chính xác là giao các tập tổ tiên, do đó kết quả là tập tổ tiên chung. Đỉnh có bit 1 với chỉ số lớn nhất (tương ứng độ sâu lớn nhất) chính là LCA.

¶ Độ phức tạp.

- Tiền xử lý $O(N^2/64)$.
- Mỗi truy vấn duyệt $O(M)$ người, mỗi bước AND **bitset** $O(N/64)$. Tổng $O(Q \cdot M \cdot N/64) \approx 10^5 \cdot 2000 \cdot 2000/64 \approx 6.25 \cdot 10^9$ phép bit. Thực tế M, Q cùng lắm 2000 nên $2000 \cdot 2000/64 \approx 62500$, $Q = 2000$ cho $1.25 \cdot 10^8$, có thể chấp nhận được trong 1s với **bitset** tối ưu.
- Sự kiện đổi nhà vẫn $O(1)$.

§3.2.4 Subtask 3 (20% số điểm, $r_1 = 1$ và $r_i = i - 1$ với $i \geq 2$)

¶ **Tiếp cận.** Cây là một đường thẳng $1 \leftarrow 2 \leftarrow \dots \leftarrow N$. Khi đó $depth(i) = i - 1$ và LCA của tập nhà chính là min các chỉ số nhà. Điều kiện hợp lệ trở thành:

$$\max(\text{nghĩa}) - \min(\text{nghĩa}) \leq R.$$

Bài toán quy về: cho một dãy vòng tròn M phần tử (nhà của mỗi người), có cập nhật đổi chỗ, tìm đoạn liên tiếp dài nhất bắt đầu từ p sao cho $\max - \min \leq R$.

¶ **Nhận xét quan trọng.** Đối với các bài có dạng vòng tròn, ta có thể nhân đôi mảng để xử lý tính vòng tròn thành đoạn thẳng. Lý do là vì trên một vòng tròn, một đoạn liên tiếp có thể vòng ngược từ điểm nối cuối mảng (ví dụ: đi từ phần tử M vòng ngược lại phần tử 1, 2). Nếu giữ nguyên độ dài M , đoạn này sẽ bị đứt gãy thành hai phần rời rạc (từ $p \dots M$ và từ $1 \dots k$), khiến các cấu trúc dữ liệu như Segment Tree không thể truy vấn liên tục trong một thao tác. Bằng cách chép mảng gốc và nối liền vào đuôi chính nó, bất kỳ một đoạn liên tiếp nào trên vòng tròn (có độ dài tối đa M) bắt đầu từ p cũng đều trở thành một đoạn thẳng liền mạch trên mảng mới (nằm gọn trong khoảng từ p đến $p + M - 1$).

Khi đó, ta nhân đôi dãy thành $H[1 \dots 2M]$ (với $H[i] = H[i - M]$ với $i > M$) và dùng **Segment Tree** lưu cặp (min, max) cho mỗi đoạn.

Với truy vấn, dùng kỹ thuật **walk** trên segment tree: từ vị trí p đi về phía phải, duy trì cur_min, cur_max , gộp dần các node; nếu gộp cả node mà vẫn thỏa $\leq R$ thì nhận cả node, nếu không thì đi vào node chi tiết hơn để tìm vị trí đầu tiên vi phạm. Tương tự cho hướng trái (walk ngược). Kết quả là độ dài lớn nhất trong hai hướng, giới hạn không vượt quá M .

¶ **Độ phức tạp.** Mỗi cập nhật / truy vấn walk $O(\log M)$. Tổng $O((M + Q) \log M)$.

§3.2.5 Subtask 4 (20% số điểm, $M \leq 100$)

¶ **Tiếp cận.** Tiền xử lý **binary lifting** để tính LCA của hai đỉnh bất kỳ trong $O(\log N)$. Trong mỗi truy vấn, thay vì dùng segment tree, ta lần lượt mở rộng đoạn về mỗi hướng, mỗi bước thêm một người, cập nhật $common = LCA(common, house[người])$ và $mxDepth = \max(mxDepth, depth(house[người]))$. Dừng ngay khi $mxDepth - depth(common) > R$. Sự kiện đổi nhà cập nhật mảng ánh xạ trong $O(1)$.

¶ **Độ phức tạp.**

- Tiền xử lý LCA: $O(N \log N)$.
- Mỗi truy vấn duyệt tối đa M người, mỗi bước mất $O(\log N)$ cho LCA. Tổng truy vấn $O(Q \cdot M \log N) \leq 10^5 \cdot 100 \cdot 17 \approx 1.7 \cdot 10^8$, có thể qua nếu cài đặt tốt.

§3.2.6 Subtask 5 (20% số điểm, không ràng buộc thêm) – Lời giải đầy đủ

¶ **Ý tưởng / Tiếp cận.** Ta cần một cấu trúc vừa trả lời nhanh truy vấn tìm biên, vừa cập nhật được khi hai người đổi nhà. Cây segment tree trên dãy vòng tròn nhân đôi là lựa chọn phù hợp, nhưng thay vì chỉ lưu min, max, mỗi node sẽ lưu hai giá trị:

- lca : LCA của tất cả các nhà trong đoạn.
- $mxDepth$: độ sâu lớn nhất trong đoạn.

Phép gộp hai node A và B :

$$merge(A, B) = (LCA(A.lca, B.lca), \max(A.mxDepth, B.mxDepth)).$$

Ta thiết kế hàm **walk** trên segment tree để mở rộng đoạn hiện tại: bắt đầu với $cur_lca = 0$, $cur_mx = -1$ (đại diện tập rỗng). Tại mỗi node, thử gộp cả node vào, nếu vẫn thỏa $mxDepth - depth(lca) \leq R$ thì nhận toàn bộ node và tiếp tục sang node kế; nếu không, ta đi sâu vào node đó để tìm chính xác vị trí đầu tiên làm điều kiện sai. Kết quả trả về

là vị trí đầu tiên **không hợp lệ**. Biên phải xa nhất là vị trí ngay trước đó, biên trái xa nhất là vị trí ngay sau đó.

Khi cần mời với mốc là người p :

- Hướng phải: bắt đầu từ p trong H' , tìm biên phải xa nhất R_excl là vị trí đầu vi phạm. Số người = $\min(R_excl - 1, p + M - 1) - p + 1$ (giới hạn không quá M).
- Hướng trái: bắt đầu từ $p + M$ (bản sao thứ hai của p), đi ngược về bằng hàm walk trái để tìm biên trái đầu tiên vi phạm L_excl . Số người = $(p + M) - \max(L_excl + 1, p + 1) + 1$.

Lưu ý: Khi cập nhật đổi nhà giữa a và b , ta cập nhật vị trí của người tương ứng trong cả hai bản sao.

¶ Các bước của thuật toán.

1. Tiền xử lý cây:

- Xác định gốc s (nơi $r_s = s$).
- Duyệt danh sách kề từ mảng r_i .
- DFS từ gốc để tính mảng $depth$ và bảng $up[j][u]$ (tổ tiên cách 2^j cạnh) cho binary lifting.

2. Xây dựng segment tree:

- Khởi tạo dãy H' độ dài $2M$: $H'[i] = \text{n nhà của người } (i - 1) \bmod M + 1$.
- Mỗi node lưu lca và $mxDepth$ của đoạn, build từ lá lên gốc bằng phép merge.

3. Xử lý sự kiện:

- Với loại 1: tìm người đang ở a và b , đổi chỗ trong mảng `person_at` và `house_of`. Cập nhật segment tree tại các vị trí tương ứng (và vị trí $+M$).
- Với loại 2: thực hiện walk phải và walk trái như mô tả, lấy max kết quả hai hướng, in ra.

¶ Chứng minh tính đúng đắn.

- Việc gộp LCA của một tập bằng cách LCA từng đôi là đúng do tính kết hợp của phép LCA: $LCA(x_1, \dots, x_k) = LCA(LCA(x_1, \dots, x_{k-1}), x_k)$. Segment tree lưu thông tin gộp của các đoạn con, nên node cha chứa LCA chính xác của cả đoạn.
- Hàm walk dựa trên tính đơn điệu: khi gộp một node vào tập hiện tại, nếu điều kiện vẫn đúng thì toàn bộ phần tử trong node đó đều hợp lệ. Nếu node lớn không hợp lệ, ta phải xét chi tiết từng lá bên trong để tìm biên chính xác.
- Kết quả số người không vượt quá M vì ta giới hạn biên trong phạm vi một vòng tròn (từ p đến $p + M - 1$ cho hướng phải, từ $p + 1$ đến $p + M$ cho hướng trái).

¶ **Ví dụ minh họa.** Xét input mẫu: $N = 8, M = 4$, cây có các cạnh... Đoạn minh họa walk: giả sử cần tìm biên phải từ $p = 1$ với $R = 1$, dãy nhà $[4, 7, 5, 8]$.

- Ban đầu $cur_lca = 0, cur_mx = -1$.
- Thêm vị trí 1 (nhà 4): $lca = 4, mx = depth[4]$. Điều kiện $mx - depth(lca) = 0 \leq 1$, OK.
- Thử gộp node lá tiếp theo (nhà 7): $lca = LCA(4, 7), mx = \max(depth[4], depth[7])$, hiệu có thể > 1 , không thỏa. Dừng, biên phải đầu vi phạm là vị trí 2, nên chỉ mời được $2 - 1 = 1$ người. Khớp output mẫu.

¶ Lưu ý về kiểu dữ liệu.

- Số người tối đa là $M \leq 10^5$, vừa tầm `int`.
- Các giá trị độ sâu, chỉ số nhà đều trong phạm vi $N \leq 10^5$, dùng `int`.
- Cẩn thận khi gộp với tập rỗng: $cur_lca = 0$ đại diện chưa có người nào. Hàm `merge_lca(u, v)` cần xử lý nếu một trong hai bằng 0 thì trả về cái còn lại.
- Khi cập nhật, nhớ cập nhật cả hai bản sao của cùng một người trong dãy $2M$.

¶ Độ phức tạp.

- Tiền xử lý cây: $O(N \log N)$.
- Khởi tạo segment tree: $O(M \log N)$ (mỗi node merge tốn $O(\log N)$).
- Mỗi cập nhật đổi nhà: $O(\log M \cdot \log N)$ (tối đa 4 điểm update, mỗi điểm ảnh hưởng $O(\log M)$ node).
- Mỗi truy vấn loại 2: $O(\log M \cdot \log N)$ (hàm walk ghé thăm $O(\log M)$ node, mỗi lần merge LCA tốn $O(\log N)$).
- Tổng thời gian: $O((N + M + Q) \log M \log N)$, đủ để chạy trong giới hạn 10^5 .
- Bộ nhớ: $O(N \log N + M)$ cho bảng binary lifting và segment tree.

¶ Mã nguồn minh họa (C++).

```

1 #include <bits/stdc++.h> //NeOWami
2 using namespace std;
3
4 const int N = 1e5 + 5;
5
6 int n, m, q;
7 int p_arr[N];
8 vector<int> adj[N];
9 int root;
10
11 int depth[N];
12 int up_arr[20][N];
13
14 int house_of[N];

```

```
15 int person_at[N];
16
17
18 int st[N * 8][2];
19
20 inline int get_lca(int u, int v) {
21     if (!u || !v) return u | v;
22     if (depth[u] < depth[v]) swap(u, v);
23     for (int i = 19; i >= 0; i--) {
24         if (depth[u] - (1 << i) >= depth[v]) {
25             u = up_arr[i][u];
26         }
27     }
28     if (u == v) return u;
29     for (int i = 19; i >= 0; i--) {
30         if (up_arr[i][u] != up_arr[i][v]) {
31             u = up_arr[i][u];
32             v = up_arr[i][v];
33         }
34     }
35     return up_arr[0][u];
36 }
37
38
39 inline int merge_lca(int u, int v) {
40     if (!u || !v) return u | v;
41     return get_lca(u, v);
42 }
43
44
45 void build(int id, int l, int r) {
46     if (l == r) {
47         int p = (l - 1) % m + 1;
48         int h = house_of[p];
49         st[id][0] = h;
50         st[id][1] = depth[h];
51         return;
52     }
53     int mid = (l + r) >> 1;
54     build(id * 2, l, mid);
55     build(id * 2 + 1, mid + 1, r);
56     st[id][0] = merge_lca(st[id * 2][0], st[id * 2 + 1][0]);
57     st[id][1] = max(st[id * 2][1], st[id * 2 + 1][1]);
58 }
59
60 void update(int id, int l, int r, int pos, int h) {
61     if (l == r) {
62         st[id][0] = h;
63         st[id][1] = depth[h];
64         return;
65     }
66     int mid = (l + r) >> 1;
67     if (pos <= mid) update(id * 2, l, mid, pos, h);
```

```
68     else update(id * 2 + 1, mid + 1, r, pos, h);
69     st[id][0] = merge_lca(st[id * 2][0], st[id * 2 + 1][0]);
70     st[id][1] = max(st[id * 2][1], st[id * 2 + 1][1]);
71 }
72
73 int walk1(int id, int l, int r, int u, int R_bound, int &cur_lca, int
    &cur_mx) {
74     if (r < u) return -1;
75     if (l >= u) {
76         int nxt_lca = merge_lca(cur_lca, st[id][0]);
77         int nxt_mx = max(cur_mx, st[id][1]);
78         if (nxt_lca == 0 || nxt_mx - depth[nxt_lca] <= R_bound) {
79             cur_lca = nxt_lca;
80             cur_mx = nxt_mx;
81             return -1;
82         }
83         if (l == r) return l;
84     }
85     int mid = (l + r) >> 1;
86     int res = walk1(id * 2, l, mid, u, R_bound, cur_lca, cur_mx);
87     if (res != -1) return res;
88     return walk1(id * 2 + 1, mid + 1, r, u, R_bound, cur_lca, cur_mx);
89 }
90
91 int walk2(int id, int l, int r, int v, int R_bound, int &cur_lca, int
    &cur_mx) {
92     if (l > v) return -1;
93     if (r <= v) {
94         int nxt_lca = merge_lca(st[id][0], cur_lca);
95         int nxt_mx = max(st[id][1], cur_mx);
96         if (nxt_lca == 0 || nxt_mx - depth[nxt_lca] <= R_bound) {
97             cur_lca = nxt_lca;
98             cur_mx = nxt_mx;
99             return -1;
100         }
101         if (l == r) return r;
102     }
103     int mid = (l + r) >> 1;
104     int res = walk2(id * 2 + 1, mid + 1, r, v, R_bound, cur_lca, cur_mx);
105     if (res != -1) return res;
106     return walk2(id * 2, l, mid, v, R_bound, cur_lca, cur_mx);
107 }
108
109 void dfs(int u, int p) {
110     up_arr[0][u] = p;
111     for (int i = 1; i < 20; i++) {
112         up_arr[i][u] = up_arr[i - 1][up_arr[i - 1][u]];
113     }
114     for (int v : adj[u]) {
115         if (v != p) {
116             depth[v] = depth[u] + 1;
117             dfs(v, u);
118         }
119     }
120 }
```

```
119     }
120 }
121
122 signed main() {
123     cin.tie(NULL)->sync_with_stdio(false);
124     if (ifstream("PARTY.inp")) {
125         freopen("PARTY.inp", "r", stdin);
126         freopen("PARTY.out", "w", stdout);
127     }
128
129     cin >> n >> m >> q;
130     for (int i = 1; i <= n; i++) {
131         cin >> p_arr[i];
132         if (p_arr[i] == i) {
133             root = i;
134         } else {
135             adj[p_arr[i]].push_back(i);
136         }
137     }
138
139     for (int i = 1; i <= m; i++) {
140         cin >> house_of[i];
141         person_at[house_of[i]] = i;
142     }
143
144     depth[root] = 0;
145     dfs(root, root);
146
147     build(1, 1, 2 * m);
148
149     while (q--) {
150         int type; cin >> type;
151         if (type == 1) {
152             int a, b; cin >> a >> b;
153             int pA = person_at[a];
154             int pB = person_at[b];
155
156             if (pA) {
157                 house_of[pA] = b;
158                 update(1, 1, 2 * m, pA, b);
159                 update(1, 1, 2 * m, pA + m, b);
160             }
161             if (pB) {
162                 house_of[pB] = a;
163                 update(1, 1, 2 * m, pB, a);
164                 update(1, 1, 2 * m, pB + m, a);
165             }
166             person_at[a] = pB;
167             person_at[b] = pA;
168         } else {
169             int p, R_bound; cin >> p >> R_bound;
170
171             int cur_lca = 0, cur_mx = -1;
```

```
172     int R_excl = walk1(1, 1, 2 * m, p, R_bound, cur_lca, cur_mx);
173     if (R_excl == -1) R_excl = 2 * m + 1;
174     int ans1 = min(R_excl - 1, p + m - 1) - p + 1;
175
176     cur_lca = 0, cur_mx = -1;
177     int end_pos = p + m;
178     int L_excl = walk2(1, 1, 2 * m, end_pos, R_bound, cur_lca,
179                       cur_mx);
180     if (L_excl == -1) L_excl = 0;
181     int ans2 = end_pos - max(L_excl + 1, p + 1) + 1;
182
183     cout << max(ans1, ans2) << "\n";
184 }
185 return 0;
186 }
```

